

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Fri 4 September 2015 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

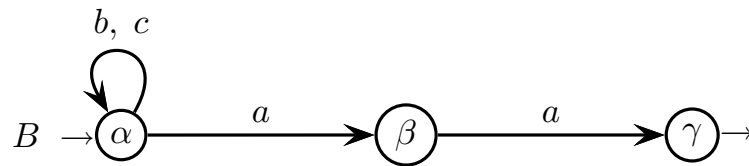
1 Regular Expressions and Finite Automata 20%

1. Given the alphabet $\Sigma = \{ a, b, c \}$, consider the following regular expression R :

$$R = c (b \mid b a)^* a$$

Answer the following questions:

- (a) Derive the deterministic automaton A that recognizes the language $L(R)$, by using the Berry-Sethi method.
- (b) Determine, in a systematic way, if the automaton A is minimal.
- (c) Consider the automaton B drawn below:



Design the automaton C that accepts the intersection of languages $L(R)$ and $L(B)$, by using the automaton cross product (Cartesian product) construction.

- (d) From the automaton C derived at the previous point, derive the regular expression R' for language $L(R) \cap L(B)$, by using the *BMC* method.
- (e) (optional) Determine if the language $L(R)$ is local and provide an adequate justification to your answer.

Solution

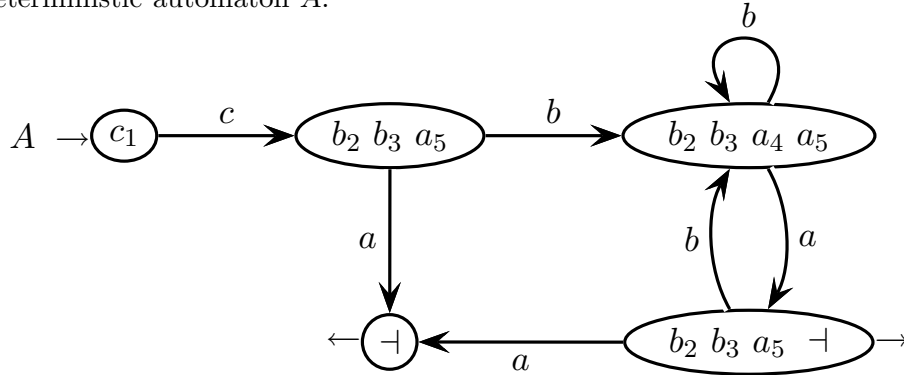
(a) Berry-Sethi. Numbered regular expression $R_{\#}$:

$$R_{\#} = c_1 (b_2 \mid b_3 a_4)^* a_5 \dashv$$

Initials and followers:

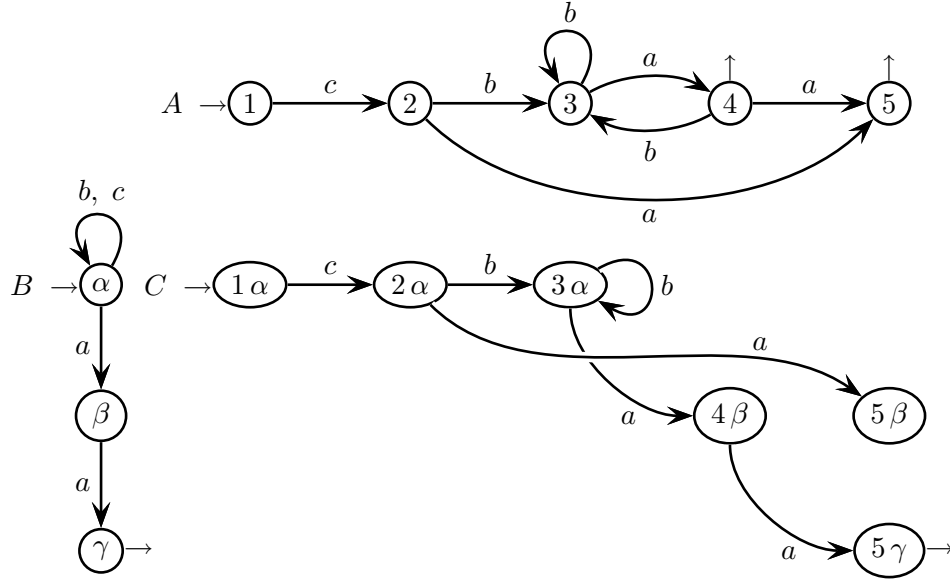
<i>initials</i>	c_1
<i>generators</i>	<i>followers</i>
c_1	$b_2 \ b_3 \ a_5$
b_2	$b_2 \ b_3 \ a_5$
b_3	a_4
a_4	$b_2 \ b_3 \ a_5$
a_5	$\dashv$

Deterministic automaton A :

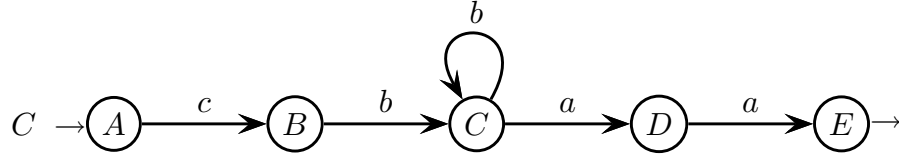


(b) Automaton A is minimal. First notice that it is clean (all the states are useful). Second: the two final states have outgoing arcs with different labels, therefore they are distinguishable; the initial state has an outgoing arc with a label different from those of the outgoing arcs of the other two non-final states, therefore it is distinguishable from such states; and the two non-initial non-final states have outgoing arcs with label a that go to two states already known to be distinguishable, therefore they are distinguishable as well. Thus all the states of A are distinguishable and in conclusion the automaton is minimal.

(c) Product construction (the state names of automaton A are shortened and the product states that are inaccessible from the initial one are not drawn):



The product automaton C is not clean, as state 5β is not post-accessible (because it does not reach any final state) and thus can be canceled. Here is the clean product automaton C , after renaming the states again:



The clean automaton C is the product of two deterministic automata, hence it is deterministic, too. It would be easy to verify that it is minimal as well.

- (d) We skip the stepwise application of the *BMC* method to the (clean) automaton C , as it is quite simple, and we directly go to the fairly intuitive result:

$$R' = c b b^* a a = c b^+ a^2$$

- (e) The language $L(R)$ is not local. In fact the local sets of $L(R)$ are $Ini(L(R)) = \{c\}$, $Fin(L(R)) = \{a\}$ and $Dig(L(R)) = \{aa, ba, bb, cb, ca\}$. They allow the string caa , yet such a string does not belong to $L(R)$.

Caveat! One sees soon that every state of automaton A has ingoing arcs with only one label, yet there are two states both with ingoing arcs labeled by letter a , namely 4 and 5. This is not a sufficient condition for the language to be local.

Remember: A sufficient condition for the language to be local, is that the states of the automaton are *characterized* by having ingoing arcs with only one label type, i.e., that all the ingoing arcs in a state have the same label and that there are no two states with the same ingoing label.

Remember also: The above condition on the states is *sufficient*, not necessary. Thus, if it is verified then it proves that the language is local, but if it is not verified then it does not necessarily prove that the language is not local.

2 Free Grammars and Pushdown Automata 20%

1. Consider the free language L , over the three-letter alphabet $\Sigma = \{ a, b, c \}$, defined as follows:

$$L = \{ x c w c y \mid x, y \in \{ a, b \}^* \wedge w^R = y x \}$$

Answer the following questions:

- (a) Write a non-extended (*BNF*) grammar G that generates the language L .
- (b) Consider the following language L' , which is the intersection of the language L with a regular language:

$$L' = L \cap (a b)^* c \Sigma^* c (a b)^*$$

Write a non-extended (*BNF*) grammar G' that generates the language L' .

- (c) (optional) Consider the language $L'' = \overline{L'}$, which is the complement of the language L' found at the previous point. Say if language L'' is regular or not, and reasonably justify your answer.
-

Solution

- (a) Since from $w^R = y x$ it follows that $w = x^R y^R$, the strings of language L are of type $x c x^R y^R c y$, that is, language L is the concatenation of two (independent) palindromes with centre. Grammar G is easy (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow P P \\ P \rightarrow a P a \\ P \rightarrow b P b \\ P \rightarrow c \end{array} \right.$$

Grammar G is *BNF* (and not ambiguous).

- (b) Clearly language L' is free as well, since the family of free languages is closed with respect to the intersection with the family of regular languages. The effect of the regular language is to force the left and right palindromes to alternate ab and ba , respectively. Here is grammar G' (axiom S):

$$G' \left\{ \begin{array}{l} S \rightarrow P_1 Q_1 \\ P_1 \rightarrow a P_2 a \\ P_2 \rightarrow b P_1 b \\ P_1 \rightarrow c \\ Q_1 \rightarrow b Q_2 b \\ Q_2 \rightarrow a Q_1 a \\ Q_1 \rightarrow c \end{array} \right.$$

or also:

$$G' \left\{ \begin{array}{l} S \rightarrow P Q \\ P \rightarrow a b P b a \\ P \rightarrow c \\ Q \rightarrow b a Q a b \\ Q \rightarrow c \end{array} \right.$$

Both versions of grammar G' are not ambiguous (and are *BNF*).

- (c) Language L'' is not regular. If L'' were so, then language L' itself would be regular, yet clearly it is not so. In fact, it can be formulated as follows:

$$L' = (ab)^h c (ba)^{h+k} c (ab)^k$$

and it is not regular because of the numerical relationships between the exponents, which cannot be obtained by purely iterative operators like star or cross.

A wrong answer: Since L' is free, then its complement L'' is not free and hence not regular either, as free languages are not closed with respect to complement.

Remember: Free languages not being closed with respect to complement, does not exclude that some of them may still have a free complement.

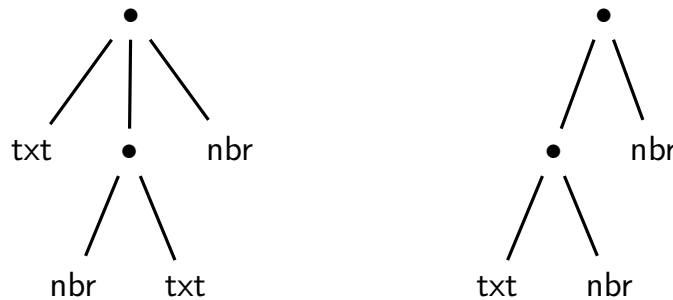
2. A marked language, inspired to *XML*, is used to encode trees in textual form. Such textual representations are subject to the following syntax constraints.

- All tree nodes are either ternary or binary inner nodes, or leaves. In particular:
 - ternary nodes have three child nodes, and are marked with the `<3TRSTRT>` and `<3TREND>` tags
 - binary nodes have two child nodes, and are marked with the `<2TRSTRT>` and `<2TREND>` tags
- Ternary nodes can have any type of child node, while the child nodes of binary nodes can only be binary nodes or leaves.
- Leaf nodes are either text elements (represented by the **txt** terminal) or number elements (represented by the **nbr** terminal).
- A phrase of the language consists of a non-empty, comma-separated, list of trees (that is, a so-called “tree forest”).

The sample text below encodes a list of two trees, the first with a ternary root and a binary inner node, and the second with a binary root and a binary inner node.

```
<3TRSTRT> txt <2TRSTRT> nbr txt <2TREND> nbr <3TREND> ,
<2TRSTRT> <2TRSTRT> txt nbr <2TREND> nbr <2TREND>
```

The figure below shows the two trees (forest) encoded by the text sample above.



Answer the following questions:

- Write an extended grammar (*EBNF*) for this marked language and verify that the grammar is not ambiguous.
- (optional) Without changing the types and specifications of the ternary, binary and leaf nodes, define a new node type with an arity (number of child nodes) strictly greater than three. This new node type has tags `<> 3TRSTART>` and `<> 3TREND>`. Its child nodes can be of any type: leaf, binary, ternary, and the new node type. Show only the rules of the previous grammar that need to be modified, and the new rules introduced (if any).

Solution

(a) Here is grammar G (axiom FOREST):

$$G \left\{ \begin{array}{l} \langle \text{FOREST} \rangle \rightarrow \langle \text{TREE} \rangle (\langle \text{' , ' } \rangle \langle \text{TREE} \rangle)^* \\ \langle \text{TREE} \rangle \rightarrow \langle \text{TERNARY} \rangle \mid \langle \text{BINARY} \rangle \mid \langle \text{LEAF} \rangle \\ \hline \langle \text{TERNARY} \rangle \rightarrow \langle \text{' < 3TRSTRT' } \rangle \langle \text{3CHILD} \rangle^3 \langle \text{' > 3TREND' } \rangle \\ \langle \text{BINARY} \rangle \rightarrow \langle \text{' < 2TRSTRT' } \rangle \langle \text{2CHILD} \rangle^2 \langle \text{' > 2TREND' } \rangle \\ \langle \text{LEAF} \rangle \rightarrow \text{txt} \mid \text{nbr} \\ \hline \langle \text{3CHILD} \rangle \rightarrow \langle \text{TERNARY} \rangle \mid \langle \text{BINARY} \rangle \mid \langle \text{LEAF} \rangle \\ \langle \text{2CHILD} \rangle \rightarrow \langle \text{BINARY} \rangle \mid \langle \text{LEAF} \rangle \end{array} \right.$$

Notice the distinction between terminals enclosed in angle brackets (outlined by apices), and nonterminals, which is essential. Notice also there are two rules (for TREE and 3CHILD) that have the same right part. Such rules could be unified, though keeping both of them helps us understand the grammar more easily. Grammar G is *EBNF* and not ambiguous, as it is designed by using non-ambiguous substructures.

(b) Extending grammar G as required is quite simple. Modified rule:

$$\langle \text{TREE} \rangle \rightarrow \langle \text{NARY} \rangle \mid \langle \text{TERNARY} \rangle \mid \langle \text{BINARY} \rangle \mid \langle \text{LEAF} \rangle$$

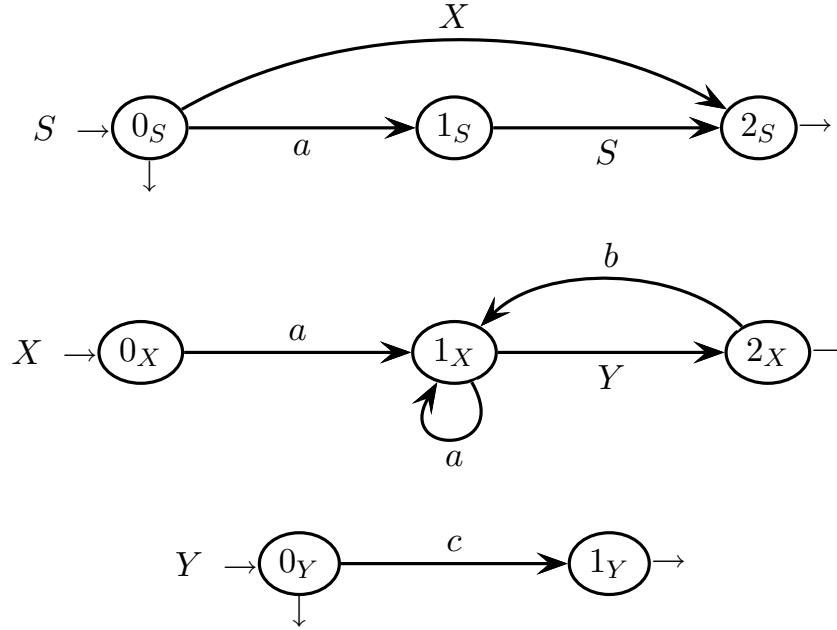
Added rules:

$$\begin{aligned} \langle \text{NARY} \rangle &\rightarrow \langle \text{' < } n \text{TRSTRT' } \rangle \langle \text{NCHILD} \rangle^n \langle \text{NCHILD} \rangle^+ \langle \text{' > } n \text{TREND' } \rangle \\ \langle \text{NCHILD} \rangle &\rightarrow \langle \text{NARY} \rangle \mid \langle \text{TERNARY} \rangle \mid \langle \text{BINARY} \rangle \mid \langle \text{LEAF} \rangle \end{aligned}$$

A digression on semantic ambiguity. Despite the apparent clarity of the clause “Without changing the types and specifications of the ternary, binary and leaf nodes ...”, it is questionable whether now a ternary node may also have n_{ary} child nodes. We chose to change grammar G as little as possible, so we opted in the negative sense. One might instead argue that (going back to the specifications), if a ternary node can have “any type of child node”, then this will comprehend also the new node type, after allowing it into the language. Both viewpoints are acceptable, as it depends on the semantic ambiguity of the indefinite adjective “any” ... We preferred to avoid any addition that was not explicitly mentioned, as hinted by the parenthesis of the “any” clause. Yet the reader may wish to write by himself a modified grammar for the viewpoint we discarded, to make more practice.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following *EBNF* grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ (axiom S):



Answer the following questions:

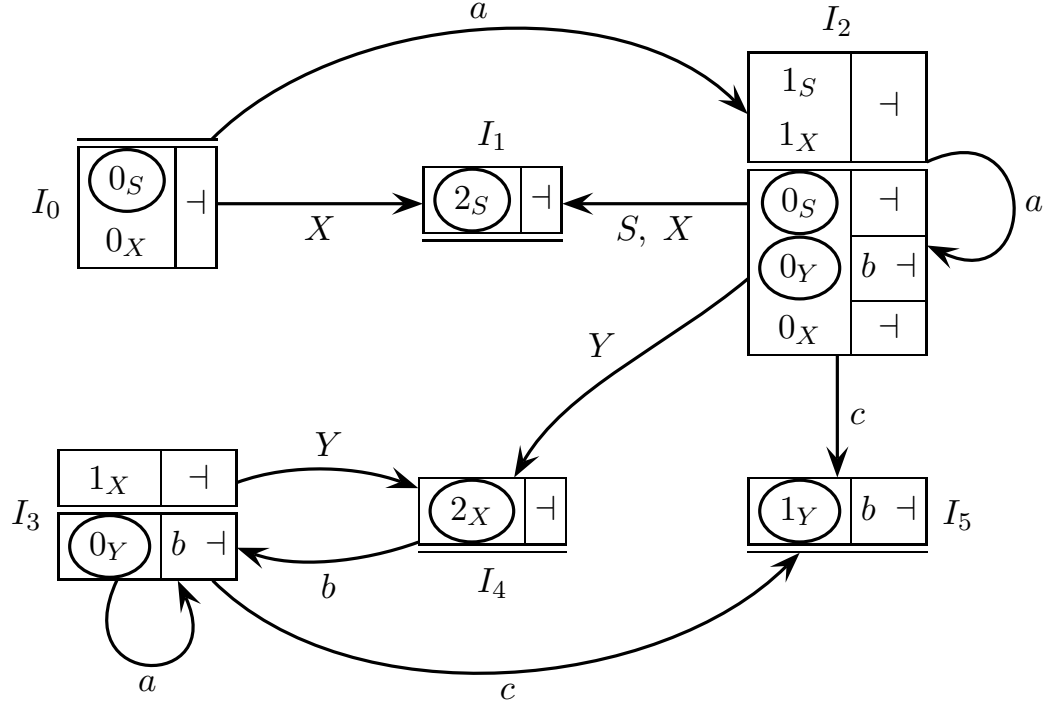
- (a) Draw the complete *ELR* pilot of grammar G , say if grammar G is of type *ELR* (1) or not, and justify your answer.
- (b) Find all the guide sets on all the arcs (terminal arcs, call arcs and exit arrows) of the machine net of grammar G , say if grammar G is of type *ELL* (1) or not, and justify your answer.
- (c) Say if grammar G is of type *ELL* (k) for some $k \geq 2$, and reasonably justify your answer.
- (d) (optional) Say if language $L(G)$ is of type *ELL* (k) for some $k \geq 2$, and reasonably justify your answer.

Solution

- (a) The pilot of grammar G is easy: it has 6 m-states and it has an RR conflict in the m-state I_2 . Thus grammar G is not $ELR(1)$. Notice also that the string a is ambiguous, as it has these two derivations:

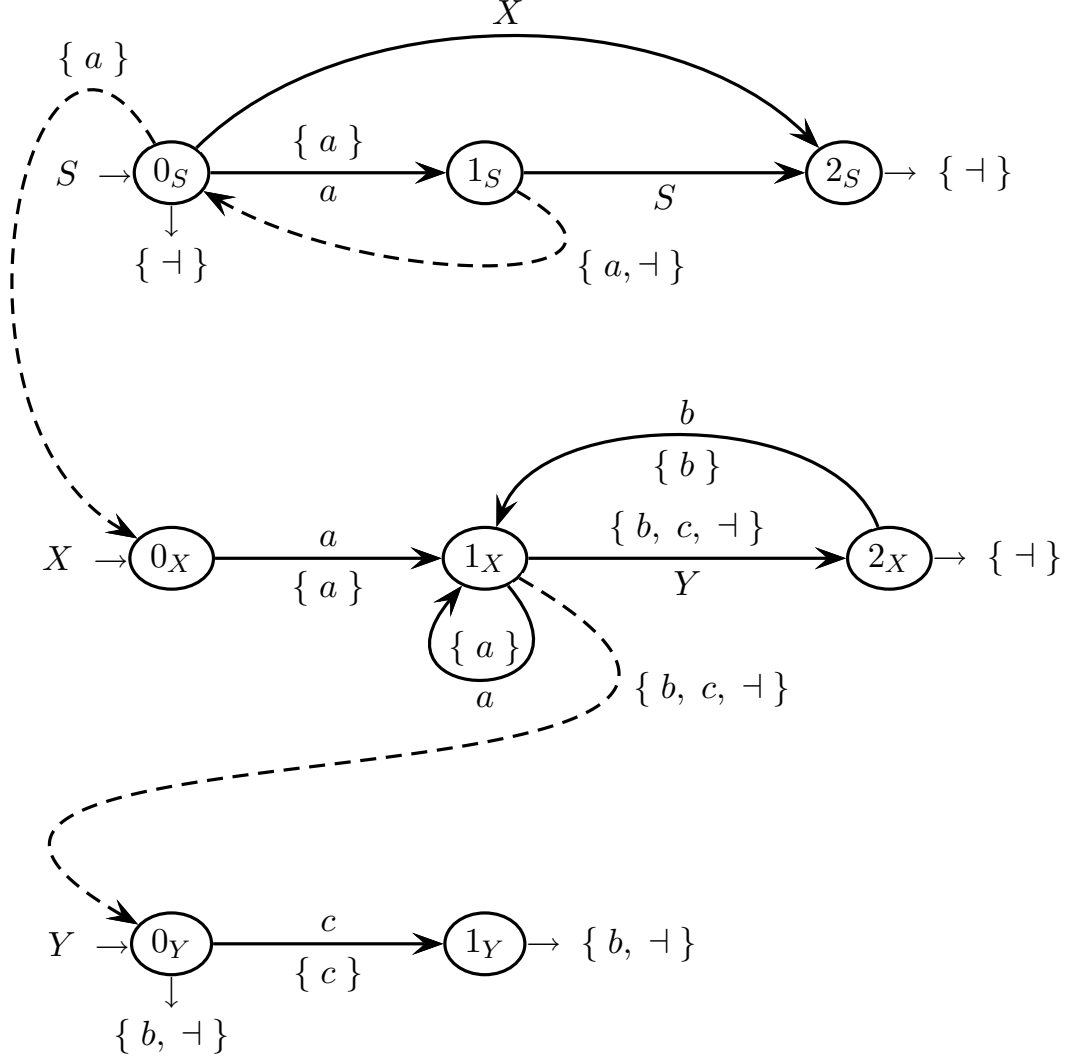
$$S \xrightarrow{S \rightarrow a S} a S \xrightarrow{S \rightarrow \varepsilon} a \varepsilon = a$$

$$S \xrightarrow{S \rightarrow X} X \xrightarrow{X \rightarrow a Y} a Y \xrightarrow{Y \rightarrow \varepsilon} a \varepsilon = a$$



By the way, the pilot of grammar G does not have the STP property, as the basis of m-state I_2 contains two candidates. In fact, transition $I_0 \xrightarrow{a} I_2$ is multiple (more precisely double, but not convergent).

(b) Here are all the call arcs and guide sets:



There are two overlapping guide sets on state 0_S with the same element a , thus grammar G is not $ELL(1)$ (and also because the pilot does not have the STP).

Caveat ! How is the guide set on the call arc $1_X \dashrightarrow 0_Y$ computed?

Correct answer: letter c is in, as c is an initial of nonterminal Y ; letter b is in, as Y is nullable and b follows Y in the machine M_X ; and the end-of-text $\vdash$ is in, as Y is at the end of M_X , whose prospect set is $\vdash$ (the shift arc $1_X \xrightarrow{Y} 2_X$ enters directly the final state 2_X and the prospect set of 2_X is $\vdash$).

Typical wrong answer: b and $\vdash$ are in, as they figure as the guide set on the exit arrow of the destination state 0_Y of the call arc $1_X \dashrightarrow 0_Y$, and therefore they back-propagate onto the call arc.

Remember: the guide set on an exit arrow of a final state does not back-propagate to the guide set of a call arc entering that final state.

A similar situation occurs for the call arc $1_S \dashrightarrow 0_S$.

- (c) Both overlapping guide sets of length k on state 0_S contain elements of type a^k for any $k \geq 1$. In fact, by lengthening the derivations shown at point (a), we obtain:

$$\begin{aligned} S &\xrightarrow{(S \rightarrow a S)^k} a^k S \xrightarrow{S \rightarrow \varepsilon} a^k \\ S &\xrightarrow{S \rightarrow X} X \xrightarrow[\text{looping on } 1_X \text{ with } a \text{ for } k-1 \text{ times}]{X \rightarrow a a^{k-1} Y} a^k Y \xrightarrow{Y \rightarrow \varepsilon} a^k \end{aligned}$$

This implies that string a^k is in the guide set of order k of both the shift arc $0_S \rightarrow 1_S$ and the call arc $0_S \dashrightarrow 0_X$. Thus grammar G is not $ELL(k)$ for any $k \geq 2$ (all the other guide sets on bifurcation states are not overlapping).

- (d) Notice: language $L(Y)$ is finite, hence regular; language $L(X)$ is regular, as the only nonterminal Y that appears in the machine M_X generates a regular (more precisely finite) language; language $L(S)$ is generated by the extended rule $S \rightarrow a S \mid X \mid \varepsilon$ (obtained by finding a regular expression for machine M_S), which is basically right-linear (the alternative $S \rightarrow X$ does not have a strictly right-linear form, but it generates a purely regular language, as argued before). Therefore such a rule can be solved by the Arden identity and yields $L(S) = a^* (L(X) \cup \varepsilon)$. Thus language $L(S)$ is regular since language $L(X)$ is regular, as argued before. Since by definition it holds $L(G) = L(S)$, the language generated by grammar G is regular, whence it is $ELL(1)$.

More in detail, by informally finding the corresponding regular expressions:

$$\begin{aligned} L(Y) &= c \mid \varepsilon \\ L(X) &= a^+ L(X) (b a^* L(X))^* \\ L(S) &= a^* (L(X) \mid \varepsilon) \end{aligned}$$

Hence, by substitution:

$$\begin{aligned} L(Y) &= c \mid \varepsilon \\ L(X) &= a^+ (c \mid \varepsilon) (b a^* (c \mid \varepsilon))^* \\ L(S) &= a^* \left(\left(a^+ (c \mid \varepsilon) (b a^* (c \mid \varepsilon))^* \right) \mid \varepsilon \right) \end{aligned}$$

Therefore language $L(G) = L(S)$ is regular. Of course, the obtained regular expression may be simplified (for instance, it might be made non-ambiguous).

4 Language Translation and Semantic Analysis 20%

1. Given a binary string w , we denote as $\overline{w}$ the string with the bits complemented, and as w_2 the string where every bit is repeated. For instance, if $w = 0100$ then $\overline{w} = 1011$ and $w_2 = 00110000$.

Consider now the following translation τ :

$$\tau \left(w (\overline{w})^R \right) = \left((\overline{w})^R \right)_2$$

For instance, if $w = 0100$ then:

$$\tau \left(w (\overline{w})^R \right) = \tau (01001101) = (1101)_2 = 11110011$$

Notice that the strings of the source language are composed of a sequence of bits followed by the same sequence complemented and reversed.

Answer the following questions:

- (a) Design the syntactic translation scheme or grammar that defines translation τ .
 - (b) Draw the syntax tree (for the translation grammar) or trees (for the translation scheme) for the example $\tau(01001101) = (1101)_2 = 11110011$.
 - (c) (optional) Determine if the translation scheme or grammar is deterministic, and reasonably justify your answer.
-

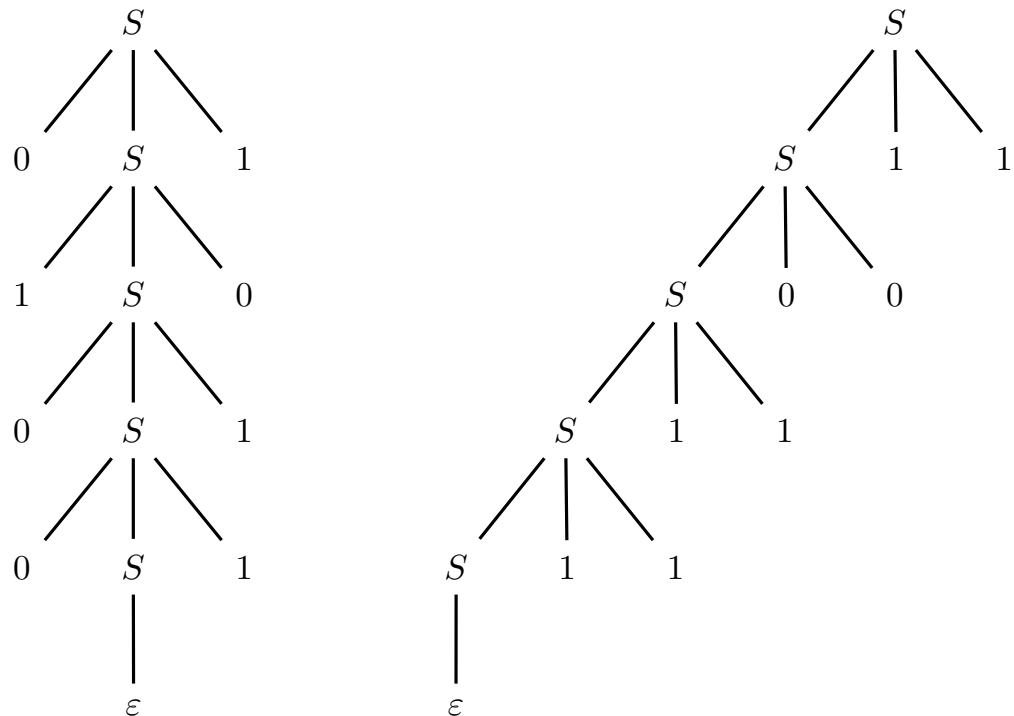
Solution

- (a) Here is the translation scheme (axiom S):

$$\begin{array}{l|l} S \rightarrow 0S1 & S \rightarrow S11 \\ S \rightarrow 1S0 & S \rightarrow S00 \\ S \rightarrow \varepsilon & S \rightarrow \varepsilon \end{array}$$

This scheme is clearly not ambiguous, so the translation associated is a function.

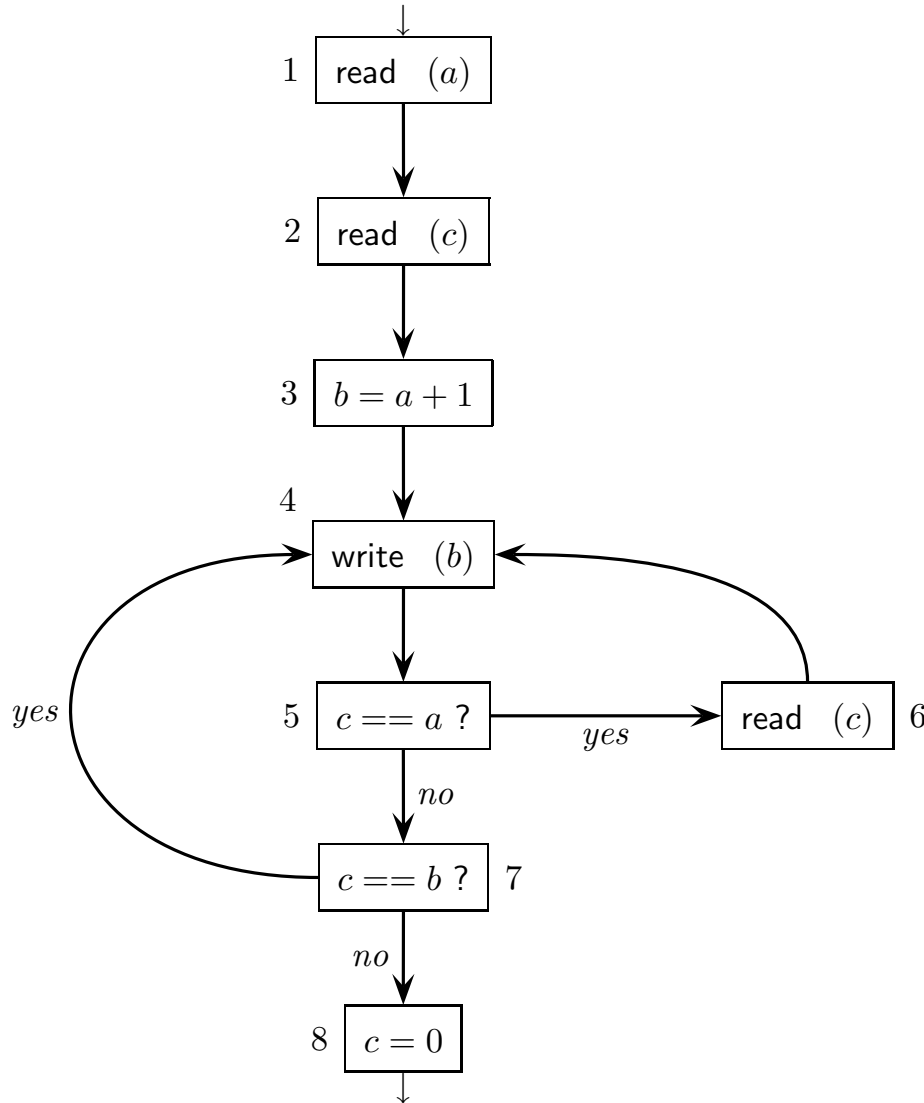
- (b) Here are the trees for the source and target grammar.



- (c) The source language is clearly nondeterministic, as it is a sort of “complemented palindrome” without a centre, well known to be inherently nondeterministic. As a consequence, the source grammar is also nondeterministic.

If we reason directly on the grammar, we easily see: that the source grammar is not LL , as we cannot predict whether to use the null rule or one of the other two (the guide sets overlap); that it is not LR , as it will have a shift-reduce conflict whether to shift on one of the two non-null rules or to reduce the null rule; and that there is not any $DPDA$ equivalent to the source grammar, as it should be able to decide whether to push onto the stack or to pop from the stack.

2. Consider the following control flow graph of a program with three integer variables a , b and c :



Answer the following questions:

- Write the regular expression R , over the node alphabet $\{1, \dots, 8\}$, that models all the program execution traces, independently of the program semantics.
- Write the system of flow equations for the live variables of the program, solve such a system iteratively and write the variables live at each node (use the tables and graph prepared on the next pages).
- (optional) By considering the program semantics, write the regular expression R' of the effective execution traces modeled by the finite automaton represented by the graph, and say what changes to the distribution of the live variables are induced by the semantics.

please here write the **REGULAR EXPRESSION R**

<i>node</i>	<i>defined</i>
1	
2	
3	
4	
5	
6	
7	
8	

<i>node</i>	<i>used</i>
1	
2	
3	
4	
5	
6	
7	
8	

system of data-flow equations for **LIVE VARIABLES**

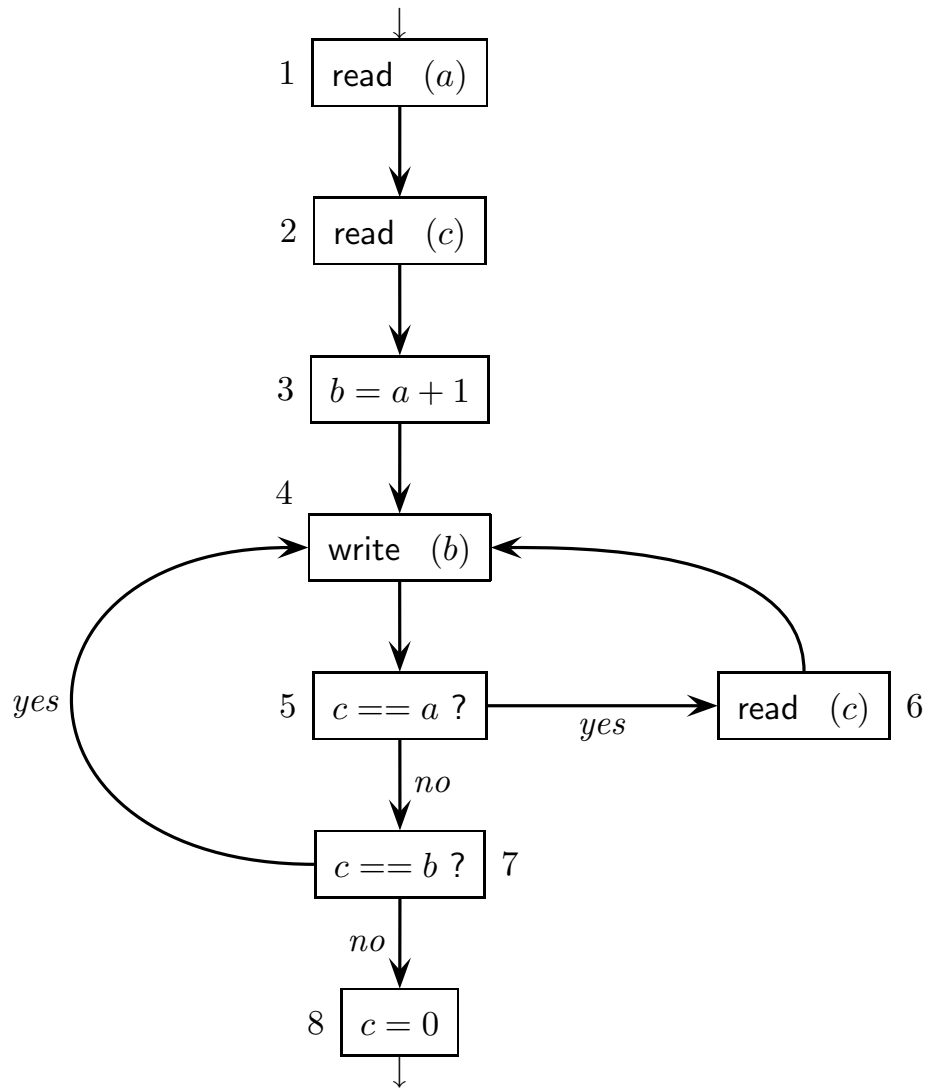
<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		
7		
8		

iterative solution table of the system of data-flow equations (**LIVE VAR.S**)
(the number of tables and columns is not significant)

	<i>initialization</i>		1		2		3		4		5		6	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														
7														
8														

	7		8		9		10		11		12		13	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														
7														
8														

please here write the **LIVE VARIABLES**



Solution

(a) Regular expression R :

$$R = 1\ 2\ 3\ (4\ 5\ (6\ |\ 7))^* 4\ 5\ 7\ 8$$

(b) Variable tables:

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1	a	1	—
2	c	2	—
3	b	3	a
4	—	4	b
5	—	5	$a\ c$
6	c	6	—
7	—	7	$b\ c$
8	c	8	—

System of data-flow equations:

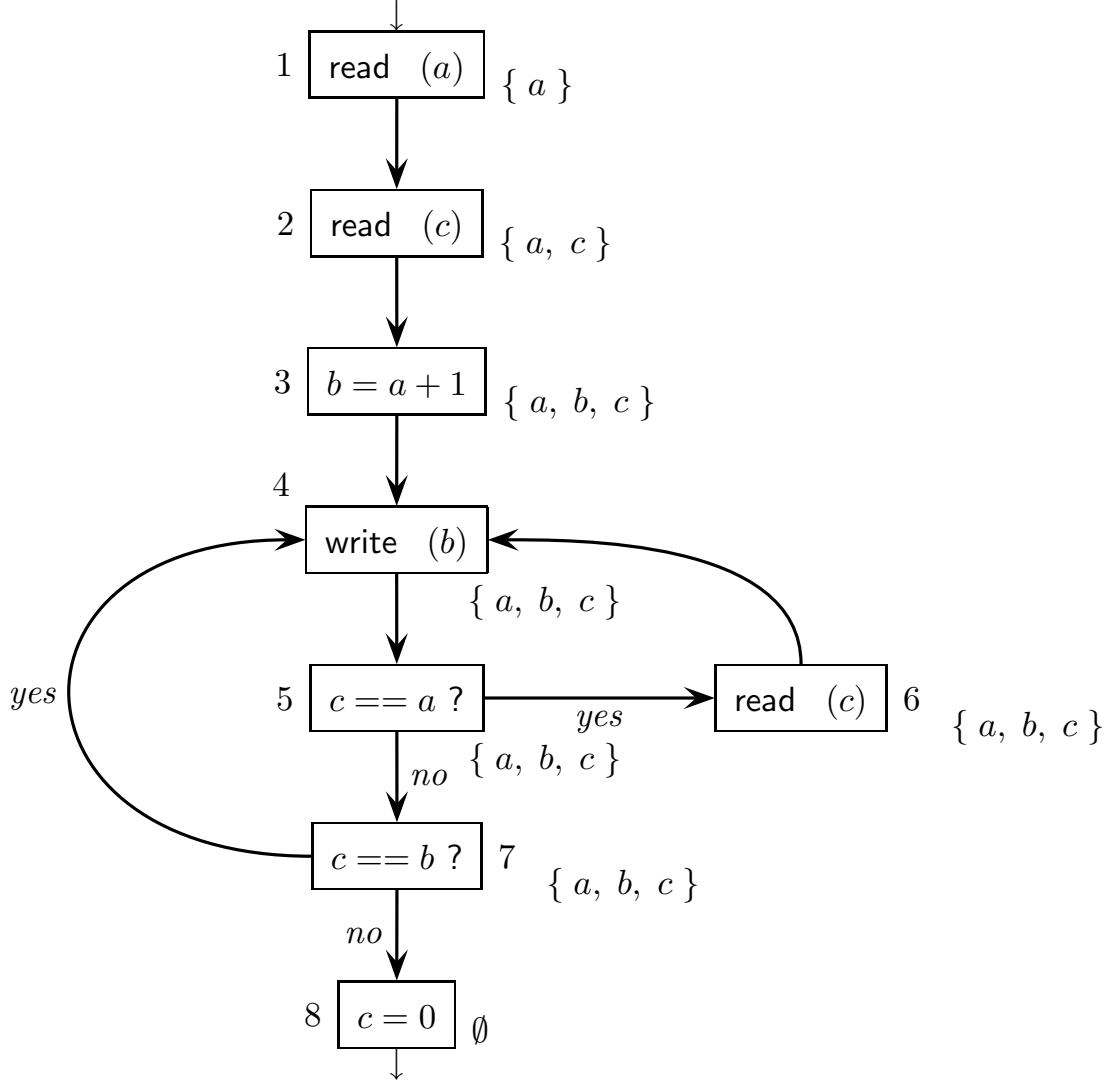
<i>node</i>	<i>in equations</i>	<i>out equations</i>
1	$in(1) = \emptyset \cup (out(1) - \{a\})$	$out(1) = in(2)$
2	$in(2) = \emptyset \cup (out(2) - \{c\})$	$out(2) = in(3)$
3	$in(3) = \{a\} \cup (out(3) - \{b\})$	$out(3) = in(4)$
4	$in(4) = \{b\} \cup (out(4) - \emptyset)$	$out(4) = in(5)$
5	$in(5) = \{a, c\} \cup (out(5) - \emptyset)$	$out(5) = in(6) \cup in(7)$
6	$in(6) = \emptyset \cup (out(6) - \{c\})$	$out(6) = in(4)$
7	$in(7) = \{b, c\} \cup (out(7) - \emptyset)$	$out(7) = in(4) \cup in(8)$
8	$in(8) = \emptyset \cup (out(8) - \{c\})$	$out(8) = \emptyset$

Iterative solution table of the system of data-flow equations:

	<i>initialization</i>		1		2		3	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	a	$\emptyset$	a	$\emptyset$
2	$\emptyset$	$\emptyset$	a	a	a	a	$a\ c$	a
3	$\emptyset$	a	b	a	$a\ b\ c$	$a\ c$	$a\ b\ c$	$a\ c$
4	$\emptyset$	b	$a\ c$	$a\ b\ c$	$a\ b\ c$	$a\ b\ c$	$a\ b\ c$	$a\ b\ c$
5	$\emptyset$	$a\ c$	$b\ c$	$a\ b\ c$	$b\ c$	$a\ b\ c$	$a\ b\ c$	$a\ b\ c$
6	$\emptyset$	$\emptyset$	b	b	$a\ b\ c$	$a\ b$	$a\ b\ c$	$a\ b$
7	$\emptyset$	$b\ c$	b	$b\ c$	$a\ b\ c$	$a\ b\ c$	$a\ b\ c$	$a\ b\ c$
8	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

The *in* columns of iteration 2 and 3 coincide, therefore the solution converges in three steps and the *out* column of iteration 3 is final.

Live variables (at the node outputs, i.e., taken from the final *out* column):



- (c) Clearly the instruction sequence 4 5 7 cannot be executed more than once, else the loop 4 5 7 is endless. This is equivalent to saying that arc $7 \xrightarrow{yes} 4$ can be ignored (though node 7 still exists and the test therein is executed). Therefore:

$$R' = 1\ 2\ 3\ (4\ 5\ 6)^* 4\ 5\ 7\ 8$$

Obviously variables a and b are no more live at (the output of) node 7.

More radically, node 7 itself becomes useless and thus it can be canceled (along with arc $7 \xrightarrow{yes} 4$), by connecting node 5 directly to node 8, i.e., $5 \xrightarrow{no} 8$. Thus the node alphabet is reduced (by removing element 7), and therefore:

$$R' = 1\ 2\ 3\ (4\ 5\ 6)^* 4\ 5\ 8$$

In this case, variable c is no more live at (the output of) node 5.